

MC-02 代码审查报告与 Bug 修复 Todolist

审查日期：2026-08-07

审查范围：Core/、bsp/、modules/、application/、USB_DEVICE/ 全部自有代码，以及 Makefile / CMakeLists 构建配置

审查方式：逐文件静态审查 + 模块间接口/中断/任务/外设交叉核对

0. 结论摘要

当前 build/ 目录存在 2026-07-26 的构建产物，但按源码分析，该固件直接上电会卡死或崩溃：

1. 工程未定义 SysTick_Handler：FreeRTOS 调度器启动后，SysTick 中断向量指向启动文件里的弱 Default_Handler（死循环），系统会在 osKernelStart() 后约 1ms 内挂死。
2. BuzzerTask() 遍历全空数组并解引用 NULL：BuzzerRegister() 从未被调用，daemon 任务第一次执行 BuzzerTask() 即 HardFault。
3. DWT 时间基准配置错误：DWT_Init(480)，而实际 HCLK 为 250MHz（500MHz SYSCLK ÷ 2），所有基于 DWT 的时间（PID dt、控制周期、CAN 超时、EKF dt）都会偏小约 1.92 倍。

除上述 3 个高优先级问题外，视觉链路（VisionUpdateTx 从未被调用、UART9 无中断）、电机控制（LK/HT/DM 多处拷贝粘贴错误）、底盘速度单位换算缺失等问题也直接影响整车功能，详见下文。

严重度定义

级别	含义
P0	必然崩溃 / 无法启动 / 核心时序错误
P1	功能失效、控制异常或大概率现场故障
P2	边界条件、健壮性、潜在越界/数据竞争
P3	建议优化、代码规范、死代码

1. Core / 系统层（main、FreeRTOS、中断、时钟）

CORE-01 [P0] 缺少 SysTick_Handler，调度器启动后系统挂死

- 位置：Core/Inc/FreeRTOSConfig.h:147（#define xPortSysTickHandler SysTick_Handler），startup_stm32h723xx.s:342（弱别名到 Default_Handler），Core/Src/stm32h7xx_it.c（全文无 SysTick_Handler）
- 现象：osKernelStart() 后第一个 tick 进入 Default_Handler 的 Infinite_Loop，所有任务无法运行。
- 修复：在 stm32h7xx_it.c 添加：

```
void SysTick_Handler(void)
{
```

```
    osSystickHandler();  
}
```

注意 HAL 时间基准已由 TIM23 提供 (HAL_TIM_PeriodElapsedCallback 中 HAL_IncTick()), 不要再在 SysTick 里重复调用 HAL_IncTick(), 否则 HAL_GetTick() 会加倍。

CORE-02 [P0] DWT 频率与实际 HCLK 不匹配

- 位置: bsp/bsp_init.h:18 (DWT_Init(480)); Core/Src/main.c (SYSCLK=500MHz, AHBCLKDivider = RCC_HCLK_DIV2 → HCLK=250MHz)
- 现象: DWT_GetDeltaT/GetTimeline* 全部按 480MHz 计算, 实际时间被压缩为 250/480≈52%。PID 积分/微分、电机离线计时、CAN 发送超时、EKF dt 全部失真。
- 修复: 改为 DWT_Init(SystemCoreClock / 1000000U) (此时 SystemCoreClock 为 250MHz), 或直接 DWT_Init(250); 同时更新 bsp_dwt.h 中“C板168MHz/A板180MHz”的注释。

CORE-03 [P2] BSPInit() 定义在头文件中

- 位置: bsp/bsp_init.h:13-22
- 说明: 非 static、非 inline 的函数定义放在头文件, 一旦被多个 .c 包含会重复定义。目前仅 robot.c 包含, 暂不报错, 建议改为 .c 实现或 static inline。

CORE-04 [P2] main.c 保留 2×32KB 未使用测试缓冲

- 位置: Core/Src/main.c:47-49 (W25Qxx_WriteBuffer/ReadBuffer, 各 32KB)
- 影响: 占用 64KB RAM (H723VG 内部 RAM 紧张), 且当前没有使用 OCTOSPI/W25Q 的代码调用它们。
- 修复: 删除或改为小缓冲/条件编译。

CORE-05 [P3] 运行时间统计回调为空实现

- 位置: Core/Src/freertos.c:74-82
- 说明: configGENERATE_RUN_TIME_STATS=1 但 configureTimerForRunTimeStats/getRunTimeCounterValue 是空实现, vTaskGetRunTimeStats 数据无意义。若不用统计功能可关闭宏。

CORE-06 [P1] FDCAN 标称波特率约为 1.0417Mbps, 不是 1Mbps

- 位置: Core/Src/fdcan.c:48-51 (FDCAN1/2/3 均为 NominalPrescaler=3, TSeg1=29, TSeg2=10)
- 计算: FDCAN 内核时钟 = PLL1Q = 125MHz (PLLM=2、PLLN=40、PLLQ=4, VCO=500MHz); 位时间 = $3 \times (1+29+10) = 120Tq \rightarrow 125\text{MHz}/120 \approx 1.0417\text{MHz}$, 与 DJI/裁判等 1Mbps 总线偏差约 4.17%, 超过 CAN 时钟容差, 可能导致电机/裁判通信误码甚至无法同步。
- 修复: 调整分频使位时间精确 1MHz, 例如 Prescaler=5, TSeg1+TSeg2=24 (125MHz/(5×25)=1MHz), 或改 PLLQ 使内核时钟为 100MHz (如 PLLQ=5) 后 Prescaler=4, 25Tq。修改后需用示波器/逻辑分析仪验证。

CORE-07 [P1] UART8/UART9 未使能 NVIC, 且中断处理函数缺失

- 位置: Core/Src/usart.c (UART8/UART9 仅配置 DMA, 无 HAL_NVIC_EnableIRQ(UART8/9_IRQn)); Core/Src/stm32h7xx_it.c (无

UART8_IRQHandler/UART9_IRQHandler)

- 现象：UART9 用于视觉 (`robot_cmd.c:58 VisionInit(&huart9)`)。
`HAL_UARTEEx_ReceiveToIdle_DMA` 依赖 IDLE 中断，缺中断后只有 DMA 收满 128 字节才会触发回调，视觉数据只能“攒够一整个缓冲”才解析一次，且多帧只取第一帧，实时性和正确性都无法保证；错误/溢出后也无法恢复。
- 修复：在 `usart.c` `MspInit` 中为 UART8/9 添加 `HAL_NVIC_SetPriority/EnableIRQ`，并在 `stm32h7xx_it.c` 添加对应 IRQ Handler。

CORE-08 [P2] I2C2 错误中断 (ER) 无处理函数

- 位置：`Core/Src/i2c.c:149` (只使能 `I2C2_EV_IRQn`)；`Core/Src/stm32h7xx_it.c` (无 `I2C2_ER_IRQHandler`)
- 影响：I2C 总线错误 (NACK/仲裁丢失) 时 HAL 无法进入错误回调，DMA/IT 传输可能永久卡住。
- 修复：使能 `I2C2_ER_IRQn` 并添加 `I2C2_ER_IRQHandler`。

CORE-09 [P2] BMI088/IST8310 的 EXTI 中断没有对应 IRQ Handler 和 NVIC 使能

- 位置：`Core/Src/gpio.c` (`ACC_INT_Pin=PE10`、`GYRO_INT_Pin=PE12` 配成 `GPIO_MODE_IT_RISING`)；`Core/Src/stm32h7xx_it.c` (无 `EXTI9_5_IRQHandler/EXTI15_10_IRQHandler`，也没有 `HAL_GPIO_EXTI_IRQHandler` 调用)
- 影响：即使启用 BMI088 新模块的触发模式，数据就绪中断也不会进入 `HAL_GPIO_EXTI_Callback`。
- 修复：按引脚使能 `EXTI9_5_IRQn/EXTI15_10_IRQn` 并添加处理函数。

2. BSP-CAN

BSPCAN-01 [P2] 过滤器数量检查应为 $\geq$ ，且三个总线计数器互相牵连

- 位置：`bsp/can/bsp_can.c:39`
- 说明：`can1_filter_idx > hfdcan1.Init.StdFiltersNbr` 在 `idx == Nbr` 时仍放行，下一个 `HAL_FDCAN_ConfigFilter` 会越界写 Message RAM；同时任一总线计数越界都会误伤其他总线注册。
- 修复：改为各总线独立判断 $\geq$ 。

BSPCAN-02 [P2] CAN 接收回调在中断里执行模块回调，缺少与任务的数据同步

- 位置：`bsp/can/bsp_can.c:160-169`
- 说明：`can_module_callback` 在 FDCAN 中断里被调用，DJI 电机等模块会直接改写 `measure` 结构体；任务 (`DJIMotorControl`) 同时在读。32 位对齐读写大体原子，但 `total_round/total_angle` 这类多步更新仍可能被撕裂，历史“编码器跳动”问题与此相关。
- 建议：回调中仅拷贝到实例缓冲并 `osSignalSet` 唤醒解析任务；或对多字更新做临界区保护。

BSPCAN-03 [P3] CANTransmit 依赖 DWT 超时，且 `busy_count` 为全局共享

- 位置：`bsp/can/bsp_can.c:207-231`
- 说明：DWT 频率错误时超时时间失真 (见 CORE-02)；`static uint32_t busy_count` 跨实例共享，日志计数无意义。建议超时改用硬件时间戳并去掉共享静态变量。

3. BSP-UART

BSPUART-01 [P1] USARTIsReady() 恒返回“忙”

- 位置: bsp/usart/bsp_usart.c:83-87

```
if (_instance->usart_handle->gState | HAL_UART_STATE_BUSY_TX)
```

- 说明: 位或结果恒非 0, 函数永远返回 0。任何依赖它做 DMA/IT 连续发送的模块将永远不发。
- 修复: 改为 (gState & HAL_UART_STATE_BUSY_TX)。

BSPUART-02 [P1] 串口错误回调未先 Abort/清错误状态就重启 DMA 接收

- 位置: bsp/usart/bsp_usart.c:127-135
- 说明: HAL 出错后 RxState 仍为 BUSY_RX, 直接 HAL_UARTEx_ReceiveToIdle_DMA 返回 HAL_BUSY, 接收永久停摆 (裁判、遥控、视觉断链后无法自恢复)。
- 修复: 先 HAL_UART_AbortReceive() (或 HAL_UART_Abort()) 再重启接收; RefereeLostCallback/RCLostCallback/VisionOfflineCallback 等离线恢复路径同样受益。

BSPUART-03 [P1] 模块回调拿不到“本次实际接收长度”

- 位置: bsp/usart/bsp_usart.c:102-118; modules/master_machine/master_process.c:82
- 说明: HAL_UARTEx_RxEventCallback 的 Size 被丢弃, 模块回调无参数; DecodeVision 只能按 recv_buff_size (固定 128) 整缓冲扫描, 会把历史残留帧当新帧解析。
- 修复: 给 usart_module_callback 增加长度参数 (或把 Size 存到 USARTInstance), 视觉/裁判等按实际长度解析。

BSPUART-04 [P3] recv_buff_size 使用 uint8_t, 缓冲上限 255

- 位置: bsp/usart/bsp_usart.h:27
- 说明: 与 USART_RXBUFF_LIMIT=256 配合时若配置 256 会截断为 0。建议改为 uint16_t 并校验。

4. BSP-SPI

BSPSPI-01 [P1] SPIDeviceOnGoing 只有索引 0 被初始化, SPI2 首次 SPITransRecv 死锁

- 位置: bsp/spi/bsp_spi.c:8

```
uint8_t SPIDeviceOnGoing[SPI_DEVICE_CNT] = {1};
```

- 说明: C 初始化器只给 [0]=1, [1] 为 0; SPITransRecv 中 while (!SPIDeviceOnGoing[1]); 会永远等待。BMI088 新模块 (modules/BMI088/bmi088.c) 通过 SPI2 收发时必然卡死。
- 修复: = {1, 1} 或改用 = { [0 ... SPI_DEVICE_CNT-1] = 1 }。

BSPSPI-02 [P1] TX 完成回调缺失, DMA/IT 纯发送后片选不释放

- 位置: bsp/spi/bsp_spi.c (只重写了 HAL_SPI_RxCpltCallback 和 HAL_SPI_TxRxCpltCallback)

- 说明：HAL_SPI_Transmit_DMA/IT 完成时调用 HAL_SPI_TxCpltCallback (HAL 弱实现为空)，CS 一直为低，后续传输失败。BMI088 写寄存器使用 SPI_Transmit，会踩中。
- 修复：实现 HAL_SPI_TxCpltCallback，复用与 RX 相同的 CS 释放逻辑。

BSPSPI-03 [P2] SPI_Transmit/SPI_Recv 不维护 SPI_DeviceOnGoing，总线互斥不完整

- 位置：bsp/spi/bsp_spi.c:36-92
- 说明：仅 SPI_Trans_Recv 做等待/置位，另外两个接口不置忙标志，多从机共享总线时可能并发拉低 CS。

5. BSP-IIC

BSPIIC-01 [P2] IIC_Register 连续 malloc 两次，第一次内存泄漏

- 位置：bsp/iic/bsp_iic.c:14-15
- 修复：删除重复的一行。

BSPIIC-02 [P1] IIC_SEQ_HOLDON 与 IIC_SEQ_RELEASE 枚举值重复（都为 0）

- 位置：bsp/iic/bsp_iic.h:27-30

```
IIC_SEQ_RELEASE,  
IIC_SEQ_HOLDON = 0,
```

- 说明：HOLD ON 模式永远无法与 RELEASE 区分，顺序传输（先 HOLDON 写地址、再 RELEASE 传数据）会提前释放总线。
- 修复：IIC_SEQ_HOLDON = 1。

BSPIIC-03 [P3] IIC_AccessMem 注释说超时 1ms，实际 100ms；无返回状态检查

- 位置：bsp/iic/bsp_iic.c:108-124
- 说明：建议检查 HAL 返回值并处理错误（配合 CORE-08 的 ER 中断）。

6. BSP 其他（DWT / PWM / GPIO / Flash / WS2812 / Log）

BSPMISC-01 [P0] DWT 频率错误（同 CORE-02，此处不再重复）

BSPMISC-02 [P2] controller.c 积分限幅使用 static 临时变量

- 位置：modules/algorithm/controller.c:40

```
static float temp_Output, temp_Iout;
```

- 说明：所有 PID 实例共享这两个变量；多个电机任务并发（HT/DM 每电机一个任务）时会互相污染限幅结果。

- 修复：改为函数内局部变量。

BSPMISC-03 [P2] `float_rounding()` 用 `static` 且负数舍入错误

- 位置：`modules/algorithm/user_lib.c:92-99`
- 说明：`static int integer / decimal` 跨调用共享；`-1.6` 返回 `-1` 而不是 `-2`。INS 温度控制会用到它。

BSPMISC-04 [P2] `AverageFilter` 对 `len==0` 无保护

- 位置：`modules/algorithm/user_lib.c:130-139`
- 说明：`len-1` 下溢。调用方目前传 5，但建议加参数校验。

BSPMISC-05 [P1] `flash_write_*` 会越界写扇区边界、静默截断且不检查擦除/编程结果

- 位置：`bsp/flash/bsp_flash.c:32-90`
- 说明：`while (uw_address <= end_address)` 在边界地址也执行一次写入；`len` 超过扇区容量时静默停止并返回 0；`flash_erase_address` 忽略 HAL 返回值；H723VG 只有 1MB（扇区 0-11），头文件中的扇区 12-23 对该芯片不存在。
- 修复：按“起始地址 + 对齐到扇区/页”重写，并检查 HAL 返回值；删除不存在扇区的定义。

BSPMISC-06 [P2] WS2812 驱动与 ADC 共用 PA5

- 位置：`bsp/ws2812/ws2812.c:19-22`（SPI6_SCK=PA5）；`Core/Src/adc.c`（ADC1_INP19=PA5）
- 说明：`WS2812_Init()` 会把 PA5 从模拟输入改配为 AF，导致 ADC1 通道 19 永久失效；两个外设不能同时使用。
- 修复：确认板级设计；若必须共存，换引脚或二选一。

BSPMISC-07 [P2] WS2812 编码时序偏快，且存在 `Size=0` 的无效发送

- 位置：`bsp/ws2812/ws2812.c:74-86`
- 说明：SPI6 用 HSE（25MHz）÷4 = 6.25MHz（注释写 24MHz/6MHz，实际 HSE=25MHz），每 WS2812 位 4 个 SPI bit ≈ 640ns，低于标准 1.25μs；`HAL_SPI_Transmit(&hspi6, &res, 0, ...)` `Size=0` 无意义。需按实际时钟核对 T0H/T1H 时序。

BSPMISC-08 [P3] `Float2Str` 用截断而非四舍五入

- 位置：`bsp/log/bsp_log.c:28-37`
- 说明：`(int)((va-head)*1000)` 会截断小数；对负数和舍入误差敏感。仅调试用，建议改用 `snprintf("%.3f")`。

7. 算法层（controller / RLS / Kalman / QuaternionEKF / user_lib）

ALG-01 [P2] PID 死区内只清输出不清积分

- 位置：`modules/algorithm/controller.c:146-149`
- 说明：进入死区后 `Iout` 保持，退出死区瞬间可能有积分冲击。建议死区内同时清零/保持策略明确化。

ALG-02 [P2] PID 微分项存在 `dt==0` 除零风险

- 位置: `modules/algorithm/controller.c:135`
- 说明: 若两个控制周期之间 DWT 未推进 (初始化后立即调用、或时钟错误), `dt` 可能为 0, `Kd*(Err-Last_Err)/dt` 产生 inf/NaN。建议 `dt` 下限保护。

ALG-03 [P2] `invSqrt(0)` 在加速度/陀螺仪全零时产生 inf/NaN

- 位置: `modules/algorithm/QuaternionEKF.c:154-164`
- 说明: 加速度计三轴全 0 时 `accelInvNorm=inf`, 量测向量变成 NaN; 陀螺仪全 0 时 `gyro_norm=1/inf=0` 尚可。建议对输入范数加最小阈值。

ALG-04 [P3] `RLS` 依赖 `RLS_Reset()` 初始化协方差, 但调用方 (power_manager) 把初始化注释掉了

- 位置: `modules/power/power_manager.c:18-27`
- 说明: 若启用功率 `RLS` 需先 `RLS_Reset`; 当前模块整体未接线, 属于未成功能。

8. 电机模块

MOTOR-01 [P1] LK9025: 角度环用电流反馈、速度环调用角度 PID、反馈源判断用错字段

- 位置: `modules/motor/LKmotor/LK9025.c:113-125`
 - 角度环 `pid_measure = measure->real_current` (应为 `total_angle`)
 - 速度环 `PIDCalculate(&motor->angle_PID, ...)` (应为 `speed_PID`)
 - 速度反馈源判断用 `angle_feedback_source` (应为 `speed_feedback_source`)
- 影响: LK 电机闭环完全不可用。

MOTOR-02 [P1] HT04: 速度反馈源判断误用角度字段

- 位置: `modules/motor/HTmotor/HT04.c:169`
- 说明: `setting->angle_feedback_source == OTHER_FEED` 应为 `speed_feedback_source`。

MOTOR-03 [P1] 达妙 DM 电机: 发送的力矩用未限幅且未反转的 `pid_ref`

- 位置: `modules/motor/DMmotor/dmmotor.c:134-146`
- 说明: `set` 做了反转和限幅, 但 `torque_des = float_to_uint(pid_ref, ...)` 用的是原始值; 超出量程时 12bit 编码回绕, 反转标志也不生效。
- 修复: 发送 `set`。

MOTOR-04 [P2] LK9025 与 IIC 一样存在双重 malloc

- 位置: `modules/motor/LKmotor/LK9025.c:55-56`

MOTOR-05 [P2] LK9025 多电机共用第一个实例的 CAN 发送通道

- 位置: `modules/motor/LKmotor/LK9025.c:142-146`
- 说明: 所有 LK 电机的指令都通过 `sender_instance` (第一个注册电机的 CAN 实例) 发送, 若电机分布在多条总线或发送 ID 不同会发错。

MOTOR-06 [P2] `servo_motor`: 按 `servo_id` 存储但按索引遍历, 未注册槽位解引用 NULL; 共享全局发送缓冲

- 位置: `modules/motor/servo_motor/servo_motor.c:37,66-85`
- 说明: `servo_motor_instance[servo->servo_id] = servo` 后 `DecodeServo` 却按 `i=0..servo_idx` 遍历, `servo_id≠i` 时读到 NULL; 多舵机共享 `servo_angle_write` 且无发送完成等待, DMA 会互相打断。

MOTOR-07 [P2] XM 电机骨架与 `bsp_can` 接口不匹配, 启用宏后无法编译

- 位置: `modules/motor/XMmotor/xmmotor.c:45-77`; `bsp/can/bsp_can.h` (`CAN_Init_Config_s` 无 `mode` 字段, `FDCAN_TxHeaderTypeDef` 无 `ExtId` 字段)
- 说明: `XM_MOTOR_ENABLE` 分支使用 `txconf.ExtId`、`config->can_init_config.mode`, 当前 `bsp_can` 均不支持; 启用后编译失败。且角度反馈使用从未更新的 `total_round`。

MOTOR-08 [P3] DJI 电机反馈解析在中断中执行滤波与多圈累加

- 位置: `modules/motor/DJImotor/dji_motor.c:119-137`
- 说明: 与 `BSPCAN-02` 相同, 建议把解析移到任务。

MOTOR-09 [P2] `DJIMotorControl` 对 `pid_ref` 直接截断为 `int16`, 无饱和判断

- 位置: `modules/motor/DJImotor/dji_motor.c:340`
- 说明: PID `MaxOut` 若被配置超过 32767 (如 30000), `(int16_t)pid_ref` 会回绕成负电流。建议先 `LIMIT_MIN_MAX(pid_ref, -16384, 16384)` 或发送前饱和。

9. IMU / INS

IMU-01 [P1] 水平姿态下 `InitQuaternion` 产生 NaN

- 位置: `modules/imu/ins_task.c:55-78`
- 说明: 加速度与重力方向重合时叉乘 `axis_rot` 为零向量, `Norm3d(axis_rot)` 除零得 NaN, 初始四元数 NaN, EKF 直接发散。机器人水平上电是最常见场景。
- 修复: `Norm3d` 前检查模长, 接近 0 时按单位四元数 (或仅绕 Z 初始化) 处理。

IMU-02 [P1] `EarthFrameToBodyFrame` 用了与 `Body→Earth` 相同的非转置矩阵

- 位置: `modules/imu/ins_task.c:214-237`
- 说明: 该函数与 `BodyFrameToEarthFrame` 完全同构, 应为旋转矩阵转置。影响 `gravity_b` 与 `MotionAccel_b` 计算 (小角度时误差小, 大俯仰/横滚时明显)。

IMU-03 [P2] BMI088 旧驱动 (`BMI088driver.c`) gyro 配置失败路径数组越界 + 死循环

- 位置: `modules/imu/BMI088driver.c:321-323`

```
write_reg_num--;  
error |= BMI088_Accel_Init_Table[write_reg_num][2];
```

- 说明：寄存器校验失败时 `write_reg_num--` 可能变成 255，用**加速度计**表索引越界，for 循环还会原地打转。

IMU-04 [P2] 上电标定约 3~12 秒且运行在 `__disable_irq()` 下

- 位置：`application/robot.c:36-60`；`modules/imu/BMI088driver.c:119-180`
- 说明：`INS_Init()`（`Gimballnit` 内）调用 `BMI088Init(&hspi2,1)` 在线标定 6000 次采样，期间全局中断关闭，上电延时可达数秒到十几秒。建议标定放任务里做或改为离线参数。

IMU-05 [P3] 新 BMI088 模块（`modules/BMI088/bmi088.c`）依赖的 `bsp_spi` 存在 BSPSPI-01/02 两个致命问题

- 说明：该模块当前未启用（`gimbal/robot_cmd` 里的 `bmi088_test` 均未调用），启用前必须先修 `bsp_spi`。

10. 遥控器 / 消息中心 / 守护进程

MSG-01 [P1] daemon： `init_count` 被覆盖、默认值失效、离线回调被反复触发

- 位置：`modules/daemon/daemon.c:17-23`

```
instance->temp_count = config->init_count == 0 ? 100 : config->init_count;
instance->temp_count = config->reload_count;    // 覆盖上一行
```

- 说明：`init_count` 完全无效；若调用方 `reload_count=0`，`temp_count=0` 立即离线。`DaemonTask` 在超时后每个周期都调用回调（日志刷屏/重复重连），应在离线沿只调一次。

MSG-02 [P1] message_center：新建话题的 `pub_registered_flag` 被写到头结点

- 位置：`modules/message_center/message_center.c:51`
- 说明：应为 `node->next_topic_node->pub_registered_flag = 1`；当前所有新建话题的发布标志永远为 0。

MSG-03 [P2] `SubGetMessage` 存在 `i = (i++) % N` 未定义行为

- 位置：`modules/message_center/message_center.c:91`
- 修复：`front_idx = (front_idx + 1) % QUEUE_SIZE`；

MSG-04 [P2] `PubPushMessage` 使用 `static` 迭代指针，多任务并发发布会互相干扰

- 位置：`modules/message_center/message_center.c:106`

MSG-05 [P3] 遥控器 `RectifyRCjoystick` 依赖结构体连续布局

- 位置：`modules/remote/remote_control.c:24-27`
- 说明：通过 `&rocker_1_ + i` 遍历 5 个 `int16`，结构体加字段就会出错。建议改为数组或逐个判断。

MSG-06 [P3] 遥控器离线回调清空数据与中断解析存在竞争

- 位置: `modules/remote/remote_control.c:91-99`
 - 说明: `RCLostCallback` 在任务里 `memset(rc_ctrl)`, 串口中断里 `sbus_to_rc` 同时在写; 建议解析函数先拷贝到局部再 `memcpy` 到公共区。
-

11. 视觉通信 (master_machine + USB VCP)

VISION-01 [P1] `VisionUpdateTx()` 从未被调用, 上行视觉数据全是 0

- 位置: `modules/master_machine/master_process.c:95-113` (定义);
`modules/imu/ins_task.c:171`、`application/cmd/robot_cmd.c:350-352` (只注释、不调用)
- 说明: `VisionSend()` 发送的 `send_data` 只有 `head` 和 `mode` 被初始化 (且 `mode` 也基本为 0), 四元数/云台角/弹速/弹量全部为 0。
- 修复: 在 `ins_task` 姿态解算完成后调用 `VisionUpdateTx(...)` 填充, 再统一由发送方调用 `VisionSend()`。

VISION-02 [P1] `VisionSend()` 被两个任务同时调用 (1kHz INS + 200Hz CMD)

- 位置: `application/robot_task.h:71`; `application/cmd/robot_cmd.c:352`
- 说明: 同一 DMA 串口并发发送, `HAL_UART_Transmit_DMA` 返回 BUSY 时直接丢包; 且 UART9 无 IDLE 中断 (CORE-07), 接收侧更差。
- 修复: 只保留一个发送点, 并增加发送队列/就绪检查。

VISION-03 [P1] UART 模式下用“缓冲大小”而不是“实际长度”解析

- 位置: `modules/master_machine/master_process.c:82`
- 说明: 见 BSPUART-03。整缓冲扫描会命中旧帧。

VISION-04 [P1] VCP 路径整体不可用: `USBCallback` 类型未定义 + `CDCInitRxbufferNcallback` 不存在 + `CDC_Receive_HS` 不回调

- 位置: `bsp/usb/bsp_usb.h:21` (未定义类型); `bsp/usb/bsp_usb.c:22` (调用不存在的函数);
`USB_DEVICE/App/usbd_cdc_if.c:264-269` (只重挂接收, 无用户回调)
- 说明: 一旦定义 `VISION_USE_VCP`, `Makefile/CMake` 都会链接失败; 即使链接通过, USB 收到的数据也永远不会交给 `DecodeVision`。
- 修复: 在 `usbd_cdc_if.c` 增加 `CDC_Receive_FS/HS` 式回调并把数据交给 `bsp_usb` 注册的 `rx_cbk`; 补 `USBCallback` typedef 与 `CDCInitRxbufferNcallback` 实现。

VISION-05 [P2] `VisionCheckCRC16/DecodeVision` 对非法长度无保护

- 位置: `modules/master_machine/master_process.c:48-75`
 - 说明: `recv_len` 若小于 2 (如 DMA 边界) 会越界; 建议统一用“包头 + 长度字段”定界。
-

12. 裁判系统

REF-01 [P1] 裁判帧长度未校验导致越界读/无限递归

- 位置: `modules/referee/rm_referee.c:51-95`

- 说明：`judge_length` 由报文长度字段决定，未与 `RE_RX_BUFFER_SIZE` 比较就做 CRC 和递归；恶意/损坏帧可越界读，甚至无限递归。
- 修复：先校验 `judge_length <= 缓冲剩余长度`，并用循环代替递归。

REF-02 [P3] 裁判系统数据段长度只取低字节

- 位置：`modules/referee/rm_referee.c:56`
- 说明：`buff[DATA_LENGTH]` 只读低 8 位，长度超过 255 的帧会算错（当前裁判协议帧一般 <255，属潜在问题）。

REF-03 [P3] RefereeSend 内嵌 `osDelay(115)`，只能在任务里调用

- 位置：`modules/referee/rm_referee.c:128-132`
- 说明：当前由 UI 任务调用没问题，但接口有隐含上下文要求；建议把限速放到任务层。

REF-04 [P3] UI 字符绘制 `vsprintf` 无长度限制

- 位置：`modules/referee/referee_UI.c:330`
- 说明：`graph->show_Data` 长度固定，格式化字符串过长会栈/结构体溢出，建议 `vsnprintf`。

13. 其他模块（超级电容 / CAN 通信 / 功率 / 报警 / TFmini / HC05 / OLED）

MISC-01 [P2] BuzzerTask 空指针崩溃（P0 级影响，见摘要）

- 位置：`modules/alarm/buzzer.c:52-78`
- 说明：`buzzer_list` 全为 NULL（无人调用 `BuzzerRegister`），`buzz->alarm_level` 直接解引用。`StartDAEMONTASK` 每 10ms 调用一次，必然 `HardFault`。
- 修复：循环内先判空；`BuzzerInit` 里默认注册一个实例，或未注册时直接返回。

MISC-02 [P3] LED 呼吸灯 `alpha` 用 `int8_t` 会溢出回绕

- 位置：`modules/alarm/led.c:72-96`
- 说明：`alpha` 超出 127 后回绕为负，呼吸不连续。`LEDTask` 目前无调用点。

MISC-03 [P2] TFmini: `IICReceive` 长度 11 超过 9 字节缓冲

- 位置：`modules/TFminiPlus/tfminiplus.c:25`
- 说明：`buf[9]`，接收 11 字节，越界 2 字节。该模块未加入 Makefile，启用前需修。

MISC-04 [P2] SuperCapRxCallback 使用全局单例指针

- 位置：`modules/super_cap/super_cap.c:13-23`
- 说明：多实例时会解析到最后一个实例；且无 `daemon` 离线检测。

MISC-05 [P3] HC05_SendData 不校验 `data_num` 上限

- 位置：`modules/bluetooth/HC05.c:46-58`
- 说明：`data_num > HC05_DATASIZE` 时写越界；IT 发送未等待上一帧完成。

MISC-06 [P2] OLED：斜线绘制 `x_ed` 赋值错误

- 位置：`modules/oled/oled.c:220`

```
(x1 <= x2) ? (x_ed = x2) : (x_ed = x2); // 应为 x_ed = x1
```

- 同时 `OLED_show_char` 对小于 0x20 的字符会下标越界，`OLED_printf` 的 `vsprintf` 无长度限制 (P2/P3)。

MISC-07 [P3] `power_manager` 为未完成骨架，接口无实际功能

- 位置：`modules/power/power_manager.c`
-

14. 应用层 (robot_cmd / gimbal / chassis / shoot / config)

APP-01 [P1] 底盘速度环参考值缺单位换算 (mm/s → 电机 deg/s)

- 位置：`application/cmd/robot_cmd.c:196-197` (vx/vy 为 mm/s 量级)；
`application/chassis/chassis.c:196-204` (直接 `DJIMotorSetRef(vt_*)`)
- 说明：DJI 电机速度反馈是转子 deg/s，参考值应为 $v(\text{mm/s}) / R(\text{mm}) \times (180/\pi) \times \text{减速比}$ 。当前直接把 mm/s 当 deg/s 用，速度环会饱和/失控。
`WHEEL_BASE/TRACK_WIDTH/RADIUS_WHEEL/REDUCTION_RATIO_WHEEL` 等参数完全没被使用。

APP-02 [P1] 键鼠模式横向速度键位错误

- 位置：`application/cmd/robot_cmd.c:217`

```
chassis_cmd_send.vy = rc_data[TEMP].key[KEY_PRESS].s * 300 -  
rc_data[TEMP].key[KEY_PRESS].d * 300;
```

- 说明：应为 $D*300 - A*300$ ，当前用 S 和 D，A 键无效，S 键同时影响 vx/vy。

APP-03 [P1] 键鼠模式不设置 `gimbal_mode`，云台可能一直 ZERO_FORCE

- 位置：`application/cmd/robot_cmd.c:213-282`
- 说明：`MouseKeySet` 未写 `gimbal_cmd_send.gimbal_mode`；若上电后直接切键盘（未先经过遥控器），模式保持初值 `GIMBAL_ZERO_FORCE`，`GimbalTask` 会持续停电机。

APP-04 [P1] 发射“单发/三发”无冷却，变成连发

- 位置：`application/shoot/shoot.c:129` (冷却判断被注释)、141-154
- 说明：`LOAD_1_BULLET` 每 5ms 周期都执行 `SetRef(total_angle + 36°)`，拨盘连续转。需恢复 `hibernate_time/dead_time` 逻辑或按沿触发。

APP-05 [P1] `LOAD_REVERSE` 分支没有设置任何参考值

- 位置: `application/shoot/shoot.c:162-166`
- 说明: 反转模式实际什么都不做, 拨盘保持上一模式速度。

APP-06 [P2] 连发射频换算用 8 而不是 `NUM_PER_CIRCLE=10`

- 位置: `application/shoot/shoot.c:157 (/ 8)`; `application/robot_def.h:37 (NUM_PER_CIRCLE 10)`
- 说明: `shoot_rate * 360 * REDUCTION_RATIO_LOADER / 8` 应除以 10 (或统一使用宏), 当前射频偏高 25%。

APP-07 [P2] 摩擦轮弹速档位全部设为 0, 默认档 30000 超出电流环限幅

- 位置: `application/shoot/shoot.c:178-191`
- 说明: 15/18/30m/s 三档摩擦轮参考值都是 0; 默认 30000 会被 `MaxOut=15000` 截断 (若没截断还会在 `int16` 处回绕)。需要按实际标定填入。

APP-08 [P2] `CHASSIS_ROTATE` 固定 `wz=4000`, 单位/量级可疑

- 位置: `application/chassis/chassis.c:180`
- 说明: 无论按 `deg/s` 还是 `rad/s` 都是异常大的值, 需按实际测试设定。

APP-09 [P3] gimbal 的 `GIMBAL_FREE_MODE` 注释说用编码器, 实际仍用 IMU 反馈

- 位置: `application/gimbal/gimbal.c:129-145`
- 说明: 行为与注释不符, 后续若真需要自由模式应切换 `MOTOR_FEED`。

APP-10 [P3] 配置框架 (`sentinel_config.c`) 已定义但应用层未接入

- 位置: `application/robot_def.c`、`application/config/sentinel_robot/sentinel_config.c`
- 说明: `RobotConfigInit/ChassisConfigFeed/...` 没有任何调用者; `gimbal/chassis/shoot` 仍使用硬编码参数, 且与 `sentinel` 配置不一致 (如 `gimbal` 用 `hcan3` 而配置用 `hcan1/2`)。二选一, 避免维护两套参数。

15. 构建系统

BUILD-01 [P1] CMake 构建会失败: `bsp/usb/bsp_usb.c` 被 `GLOB` 收录, 但 `USBCallback` 类型未定义

- 位置: `CMakeLists.txt:79-83 (GLOB_RECURSE "bsp/*.c")`; `bsp/usb/bsp_usb.h:21`
- 说明: `Makefile` 刻意未收录 `bsp_usb.c`, 而 `CMake` 递归包含后编译 `bsp_usb.c` 会报 `USBCallback` 未定义 (见 `VISION-04`)。

BUILD-02 [P2] Makefile 与 CMake 源文件集合不一致

- 说明: `Makefile` 未包含 `bsp/usb/`、`modules/TFminiPlus/`、`modules/ist8310/`、`modules/standard_cmd/`、`modules/unicomm/`; `CMake` 递归包含全部。两边产物行为不一致, 建议统一源文件清单。

BUILD-03 [P3] Makefile `clean` 使用 `rd /s/q`, 跨平台性差

- 位置: **Makefile** (clean 目标)
-

16. 修复 Todolist

P0 (必须先修, 否则无法运行)

- ☐ **CORE-01** 补 **SysTick_Handler** (**Core/Src/stm32h7xx_it.c**)
- ☐ **CORE-02** **DWT_Init(SystemCoreClock/1000000)**, 统一 250MHz (**bsp/bsp_init.h**)
- ☐ **MISC-01** **BuzzerTask** 判空/默认注册实例 (**modules/alarm/buzzer.c**)

P1 (直接影响功能, 建议尽快修)

- ☐ **CORE-06** FDCAN 波特率校准为 1Mbps (**Core/Src/fdcan.c**)
- ☐ **CORE-07** UART8/9 使能 NVIC + 补 IRQ Handler
- ☐ **CORE-08** 补 I2C2 ER 中断
- ☐ **BSPUART-01** **USARTIsReady** 位或改位与
- ☐ **BSPUART-02** 串口错误回调先 Abort 再重启接收
- ☐ **BSPUART-03** 回调携带实际接收长度, 视觉/裁判按长度解析
- ☐ **BSPSPI-01** **SPIDeviceOnGoing** 全部初始化为 1
- ☐ **BSPSPI-02** 实现 **HAL_SPI_TxCpltCallback** 释放 CS
- ☐ **BSPIIC-02** **IIC_SEQ_HOLDON=1**
- ☐ **MOTOR-01/02/03** 修复 LK/HT/DM 电机反馈源与力矩发送逻辑
- ☐ **IMU-01** **InitQuaternion** 处理水平零向量
- ☐ **IMU-02** **EarthFrameToBodyFrame** 使用转置矩阵
- ☐ **MSG-01** daemon 初始化与离线回调逻辑
- ☐ **MSG-02** message_center 新建话题 flag 赋值位置
- ☐ **VISION-01/02/03** 视觉上行数据填充、单点发送、实际长度解析
- ☐ **VISION-04** 修复 bsp_usb/USBCallback/CDC 回调 (若启用 VCP)
- ☐ **REF-01** 裁判帧长校验, 防越界/递归
- ☐ **APP-01** 底盘 mm/s → deg/s 换算 (使用 wheel_measure)
- ☐ **APP-02/03** 键鼠横向键位、键盘模式 gimbal_mode
- ☐ **APP-04/05/06** 发射单发冷却、反转、射频换算
- ☐ **BUILD-01** CMake 源文件清单修正 (去掉/修复 bsp_usb.c)

P2 (边界/健壮性)

- ☐ **BSPCAN-01** 过滤器计数 $\geq$ 且分总线判断
- ☐ **BSPCAN-02** CAN 回调与任务数据同步
- ☐ **BSPSPI-03** SPI 发送/接收也维护总线忙标志
- ☐ **BSPIIC-01** 删除重复 malloc
- ☐ **BSPMISC-02** PID 积分限幅 static 改局部
- ☐ **BSPMISC-03** **float_rounding** 负数舍入与 static
- ☐ **BSPMISC-05** Flash 写入按扇区边界重写并检查返回值
- ☐ **BSPMISC-06/07** WS2812 与 ADC 引脚冲突、SPI 时序核对
- ☐ **ALG-01/02/03** PID 死区积分、dt 除零、EKF invSqrt(0)
- ☐ **MOTOR-04/05/06** LK 双重 malloc、多电机发送、servo 遍历/共享缓冲
- ☐ **MOTOR-09** DJI 电流 int16 饱和

- ☐ **IMU-03** BMI088 gyro 错误路径数组越界/死循环
- ☐ **MISC-03/04/06** TFmini 缓冲、超级电容单例、OLED 斜线/printf
- ☐ **APP-07/08** 摩擦轮档位、自旋 wz
- ☐ **BUILD-02** Makefile/CMake 源文件统一

P3 (建议/优化)

- ☐ **CORE-03/04/05** BSPInit 移出头文件、删除 64KB 测试缓冲、运行统计
- ☐ **BSPMISC-08** `Float2Str` 四舍五入
- ☐ **MOTOR-08** DJI 反馈解析移出中断
- ☐ **MSG-05/06** 遥控器布局/竞争
- ☐ **REF-02/03/04** 裁判长度、发送限速、UI vsnprintf
- ☐ **APP-09/10** gimbal 模式注释对齐、配置框架接入或删除

17. 建议的修复顺序

1. 先修 P0 三项 (SysTick、DWT、Buzzer)，保证能启动、时间正确、不崩溃。
2. 再修 CAN/UART/SPI 底层 (波特率、中断、DMA 重启、SPI CS)，保证通信可靠。
3. 然后修电机与 INS 闭环逻辑 (LK/HT/DM、四元数初始化、坐标变换)。
4. 最后接视觉与裁判链路 (VisionUpdateTx、长度解析、帧校验)，并统一 Makefile/CMake 构建。

每修完一项建议同步更新本文件的勾选框，并保留修改前后的行为说明，方便回归。